



PERTEMUAN 10

Polymorphism and Interfaces

Pemrograman Berorientasi Objek – Universitas Putera Batam



Pengenalan: Konsep Polimorfisme dalam OOP

Polimorfisme memungkinkan pemrograman secara umum dengan menangani berbagai objek melalui referensi superclass yang sama. Ini adalah salah satu pilar utama dalam Pemrograman Berorientasi Objek.



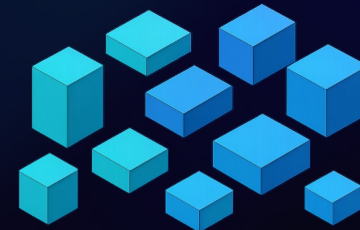
Superclass Umum

Superclass **Hewan** mendefinisikan metode `move()` dan posisi koordinat x-y. Subclass seperti **Ikan**, **Katak**, dan **Burung** mengimplementasikan `move()` sesuai perilaku masing-masing.



Penyederhanaan Kode

Objek dari subclass dapat diproses seolah-olah berasal dari superclass yang sama, sehingga kode menjadi lebih ringkas, mudah dipelihara, dan mudah diperluas.



Array Polimorfik

Array bertipe `Hewan[]` dapat menyimpan objek dari berbagai subclass dan memanggil `move()` secara dinamis berdasarkan jenis objek yang sesungguhnya pada saat runtime.

Contoh Polimorfisme: Segiempat

Hierarki kelas **Segiempat** adalah contoh klasik penerapan polimorfisme dalam desain berorientasi objek. Setiap subclass mewarisi dan mengimplementasikan operasi yang sama dengan perilaku yang berbeda.

PersegiPanjang

Subclass langsung dari Segiempat. Mengimplementasikan perhitungan keliling dan luas berdasarkan panjang dan lebar.

Persegi

Kasus khusus PersegiPanjang di mana semua sisi sama panjang. Dapat memperbarui logika perhitungan sesuai kebutuhan.

JajarGenjang

Menghitung luas berdasarkan alas dan tinggi. Diproses secara polimorfik melalui referensi Segiempat.

Trapesium

Mengimplementasikan rumus luas tersendiri namun tetap dipanggil melalui antarmuka Segiempat yang seragam.

☐ Semua objek subclass dapat diproses secara polimorfik melalui referensi **Segiempat**, tanpa perlu mengetahui jenis objek yang sebenarnya.



Contoh Kode Polymorphic

Berikut adalah ilustrasi bagaimana polimorfisme bekerja dalam praktik. Referensi superclass digunakan untuk menampung dan memanggil metode pada objek subclass yang berbeda-beda.

Deklarasi Array Polimorfik

```
Hewan[] hewanArray = new Hewan[3];  
hewanArray[0] = new Ikan();  
hewanArray[1] = new Katak();  
hewanArray[2] = new Burung();
```

Array bertipe `Hewan[]` menyimpan objek dari berbagai subclass.

Pemanggilan Metode Dinamis

```
for (Hewan h : hewanArray) {  
    h.move(); // dispatch dinamis  
}
```

Java secara otomatis memanggil implementasi `move()` yang tepat sesuai jenis objek sebenarnya pada saat runtime – inilah inti dari **dynamic method dispatch**.



Referensi
Superclass

Identifikasi
Jenis

Eksekusi
Subclass

Abstract Classes dan Methods

Kelas abstrak adalah fondasi dari hierarki pewarisan yang baik. Berbeda dengan kelas biasa, kelas abstrak dirancang sebagai kerangka dasar yang tidak dapat diinstansiasi secara langsung.

1 Tidak Dapat Diinstansiasi

Kelas abstrak dideklarasikan dengan kata kunci `abstract` dan tidak dapat dibuat objeknya secara langsung menggunakan `new`.

2 Superkelas dalam Hierarki

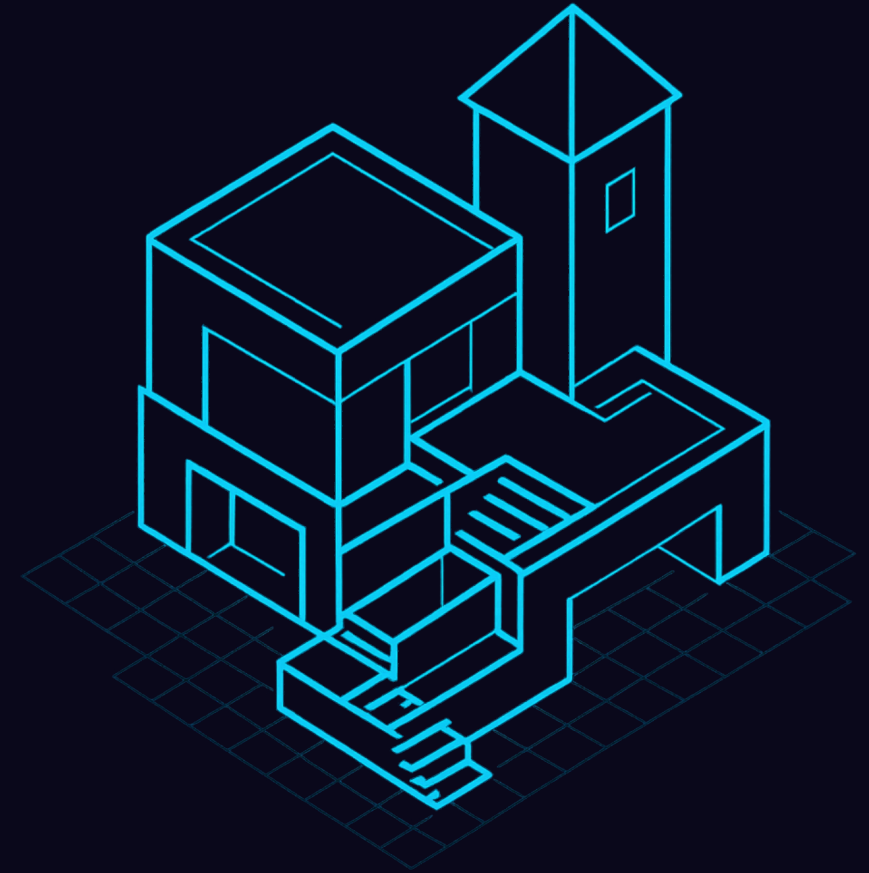
Digunakan sebagai superkelas abstrak dalam hierarki pewarisan. Kelas ini mendefinisikan kontrak yang harus dipenuhi oleh semua subclass-nya.

3 Kerangka untuk Subclass

Berfungsi sebagai kerangka dasar yang memuat metode abstrak (tanpa implementasi) maupun metode konkret yang diwarisi oleh seluruh subclass.

4 Metode Abstrak

Metode abstrak hanya memiliki deklarasi tanpa body. Setiap subclass konkret **wajib** mengimplementasikan semua metode abstrak yang diwarisi.





STUDI KASUS

Payroll System dengan Polymorphism

Sistem penggajian (payroll) adalah studi kasus klasik yang mendemonstrasikan kekuatan polimorfisme dalam aplikasi dunia nyata. Setiap jenis karyawan memiliki cara perhitungan penghasilan yang berbeda, namun semuanya dapat diproses melalui antarmuka yang seragam.



Karyawan Gaji Tetap

Penghasilan dihitung berdasarkan gaji mingguan tetap (`weeklySalary`) tanpa mempertimbangkan jam kerja aktual.



Karyawan Per Jam

Penghasilan dihitung dari tarif per jam $\times$ jumlah jam kerja, dengan tambahan 1,5 $\times$ untuk jam lembur di atas 40 jam seminggu.



Karyawan Komisi

Penghasilan berdasarkan total penjualan (`grossSales`) dikalikan tingkat komisi (`commissionRate`).



Komisi + Gaji Pokok

Menggabungkan gaji pokok (`baseSalary`) dengan hasil perkalian penjualan dan tingkat komisi untuk fleksibilitas kompensasi.

Antarmuka Umum di Java API

Java menyediakan berbagai antarmuka bawaan dalam API-nya yang digunakan secara luas. Memahami antarmuka-antarmuka ini sangat penting karena menjadi dasar dari banyak operasi fundamental di Java.



Comparable

Mendefinisikan metode `compareTo()` untuk membandingkan dua objek. Digunakan oleh `Collections.sort()` dan `Arrays.sort()`.



Iterable

Memungkinkan objek digunakan dalam enhanced for-loop. Kelas yang mengimplementasikan antarmuka ini harus menyediakan `iterator()`.



Cloneable

Menandai bahwa objek dari kelas tersebut dapat diduplikasi (di-clone) menggunakan metode `Object.clone()`.



Serializable

Memungkinkan objek dikonversi menjadi stream byte untuk disimpan ke file atau dikirim melalui jaringan.



JAVA SE 8

Interface Enhancements di Java SE 8

Java SE 8 membawa perubahan signifikan pada cara antarmuka didefinisikan dan digunakan. Tiga fitur utama diperkenalkan yang memperluas kapabilitas antarmuka secara drastis:

1

Default Interface Methods

Metode dengan implementasi konkret di dalam antarmuka. Memberikan perilaku bawaan tanpa memaksa semua implementor untuk menyediakan logika baru.

2

Static Interface Methods

Metode statis di dalam antarmuka yang berfungsi seperti metode utilitas. Tidak perlu kelas pembantu terpisah untuk memproses objek antarmuka.

3

Functional Interfaces

Antarmuka dengan tepat satu metode abstrak. Menjadi fondasi ekspresi lambda dan pemrograman fungsional di Java SE 8.

Default Interface Methods

Apa itu Default Method?

Sejak Java SE 8, antarmuka tidak hanya mendeklarasikan metode abstrak, tetapi juga dapat berisi **metode default publik** dengan implementasi konkret menggunakan kata kunci `default`.

Metode default menyediakan perilaku bawaan yang akan digunakan jika kelas implementor tidak melakukan override.

Manfaat Utama

- Antarmuka dapat **berevolusi** tanpa memaksa semua kelas turunan untuk langsung mengimplementasikan metode baru
- Menjaga **kompatibilitas ke belakang** (backward compatibility) dalam struktur kode yang sudah ada
- Mengurangi duplikasi kode pada kelas-kelas yang memiliki logika serupa
- Kelas implementor tetap dapat melakukan override jika diperlukan perilaku yang berbeda

```
interface Kendaraan {  
    default void nyalakanMesin() {  
        System.out.println("Mesin dinyalakan (default)");  
    }  
}
```

Static Interface Methods

Java SE 8 memperkenalkan metode statis di dalam antarmuka, mengubah praktik pengembangan yang telah lama berlaku sebelumnya.

Sebelum Java SE 8

Antarmuka tidak dapat memiliki implementasi metode sama sekali. Solusinya adalah membuat **kelas utilitas terpisah** yang berisi metode-metode statis pembantu (helper/utility class) untuk memproses objek yang mengimplementasikan antarmuka.

Contoh: `Collections` adalah kelas utilitas untuk antarmuka `Collection`.

Sejak Java SE 8

Metode statis dapat langsung dideklarasikan di dalam antarmuka itu sendiri. Ini **menghilangkan kebutuhan kelas utilitas terpisah** dan menjaga logika pembantu tetap dekat dengan kontrak antarmuka.

```
interface Validator {  
    static boolean isValid(String s) {  
        return s != null && !s.isEmpty();  
    }  
}
```

Private Interface Methods

Java SE 9 melanjutkan evolusi antarmuka dengan memperkenalkan metode privat, memberikan kontrol akses yang lebih granular di dalam antarmuka.

Konsep Dasar

Seperti metode helper privat di dalam kelas biasa, metode privat di antarmuka hanya dapat dipanggil oleh metode lain **di dalam antarmuka yang sama**. Metode ini tidak terekspos ke kelas implementor.

Private Instance Methods

Metode instan privat di antarmuka hanya dapat dipanggil secara langsung oleh **metode instan lain** (termasuk default methods) di dalam antarmuka tersebut. Tidak dapat dipanggil dari luar.

Private Static Methods

Metode statis privat di antarmuka dapat dipanggil oleh **metode instan maupun statis** lainnya di dalam antarmuka yang sama. Berguna untuk berbagi logika di antara default dan static methods.



Fitur ini menghindari duplikasi kode di antara beberapa metode default dalam satu antarmuka, tanpa mengekspos detail implementasi ke kelas luar.

Private Constructors pada Java

Constructor privat adalah teknik desain penting dalam Java yang membatasi bagaimana dan kapan objek dari suatu kelas dapat dibuat. Ada tiga skenario utama penggunaannya:



Preventing Object Instantiation

Mencegah pembuatan objek secara langsung dari luar kelas. Biasanya digunakan pada kelas utilitas yang hanya berisi metode statis, seperti kelas `Math` di Java, sehingga tidak ada instans yang dibuat.



Sharing Initialization Code

Berbagi logika inisialisasi di antara beberapa constructor publik dalam kelas yang sama. Constructor privat bertindak sebagai constructor pembantu yang dipanggil oleh constructor lain menggunakan `this()`.



Factory Methods

Digunakan bersama pola desain **Factory Method**. Constructor dibuat privat, dan objek hanya dapat dibuat melalui metode statis factory yang memberikan kontrol lebih besar atas proses pembuatan objek.

Program to an Interface, Not an Implementation

Prinsip ini adalah salah satu pedoman desain paling fundamental dalam OOP. Dengan memprogram terhadap antarmuka, kode menjadi lebih fleksibel dan mudah diubah.

Keterbatasan extends (Implementation Inheritance)

Java tidak mengizinkan sebuah kelas untuk mewarisi dari lebih dari satu superclass (*single inheritance*). Pewarisan implementasi melalui `extends` membawa seluruh variabel instan dan implementasi metode dari superclass.

Interface Mewarisi Interface

Sebuah antarmuka juga dapat memperluas (`extends`) satu atau lebih antarmuka lainnya, membangun hierarki kontrak yang terstruktur. Sejak Java SE 8, antarmuka juga dapat memuat implementasi metode default.

Keunggulan Interface Inheritance

Dengan `implements`, sebuah kelas dapat mengimplementasikan **beberapa antarmuka** sekaligus, di samping memperluas satu superclass. Ini memberikan fleksibilitas desain yang jauh lebih besar.



Implementation Inheritance: Tightly Coupled Classes

Kapan Menggunakan Pewarisan Implementasi?

Pewarisan implementasi paling efektif digunakan untuk mendeklarasikan kelas-kelas yang **saling terkait erat** dan memiliki banyak variabel instan serta implementasi metode yang sama.

Relasi **is-a** harus berlaku: di mana pun objek superclass diharapkan, objek subclass dapat disediakan sebagai penggantinya (Liskov Substitution Principle).

Karakteristik Kelas yang Tightly Coupled

- Berbagi banyak **variabel instan** yang sama dari superclass
- Memiliki banyak **implementasi metode** yang diwarisi tanpa perubahan
- Hubungan is-a yang kuat dan jelas antara subclass dan superclass
- Tidak memerlukan fleksibilitas untuk mengimplementasikan dari multiple sumber

📄 Contoh: `PersegiPanjang extends Segiempat` – Persegi panjang adalah segiempat, mewarisi atribut posisi dan metode dasar geometri.

Alasan Menggunakan Interface Inheritance

Meskipun membutuhkan lebih banyak pekerjaan dibandingkan pewarisan implementasi, interface inheritance menawarkan keunggulan desain yang signifikan dalam jangka panjang.



Fleksibilitas Tinggi

Kelas dapat mengimplementasikan banyak antarmuka sekaligus, memungkinkan desain yang lebih modular dan adaptif terhadap perubahan kebutuhan.



Kontrak yang Jelas

Antarmuka mendefinisikan kontrak eksplisit yang harus dipenuhi setiap implementor, sehingga lebih mudah dipahami dan diikuti oleh pengembang lain.



Implementasi Manual

Setiap kelas harus menyediakan implementasi untuk semua metode abstrak, bahkan jika logikanya serupa. Ini memerlukan usaha lebih namun menghasilkan kontrol penuh.



Loosely Coupled

Kelas implementor tidak terikat pada hierarki pewarisan tunggal, sehingga lebih mudah untuk diuji, direfaktorkan, dan diganti tanpa mempengaruhi bagian lain dari sistem.

Hirarki Aplikasi Karyawan — Bagian 1

Interface `CompensationModel` mendefinisikan kontrak tunggal: metode abstrak `penghasilan()`. Setiap model kompensasi mengimplementasikan metode ini dengan cara perhitungan yang berbeda.

CompensationModel

Interface dengan satu metode abstrak:
`double penghasilan()`
Diimplementasikan oleh semua model kompensasi karyawan.

SalariedCompensationModel

Variabel: `weeklySalary`
`penghasilan()` mengembalikan nilai `weeklySalary` secara langsung tanpa perhitungan tambahan.

HourlyCompensationModel

Variabel: `wage`, `hours`
`penghasilan()` menghitung upah × jam kerja, dengan tambahan **1,5×** upah untuk jam lembur di atas 40 jam.

Hirarki Aplikasi Karyawan — Bagian 2

Dua model kompensasi berbasis komisi melengkapi hierarki, menunjukkan bagaimana antarmuka yang sama dapat digunakan untuk logika bisnis yang beragam.

CommissionCompensationModel

Variabel: `grossSales`, `commissionRate`

Rumus penghasilan():

```
grossSales * commissionRate
```

Seluruh penghasilan bergantung pada kinerja penjualan tanpa gaji pokok.

BasePlusCommissionModel

Variabel: `grossSales`, `commissionRate`, `baseSalary`

Rumus penghasilan():

```
baseSalary + (grossSales * commissionRate)
```

Menggabungkan keamanan gaji pokok dengan insentif berbasis kinerja.

- ❏ Melalui polimorfisme, semua model ini dapat diproses dalam satu loop dengan memanggil `penghasilan()` tanpa perlu mengetahui jenis model yang digunakan.

Studi Kasus GUI & Graphics: Menggambar dengan Polymorphism

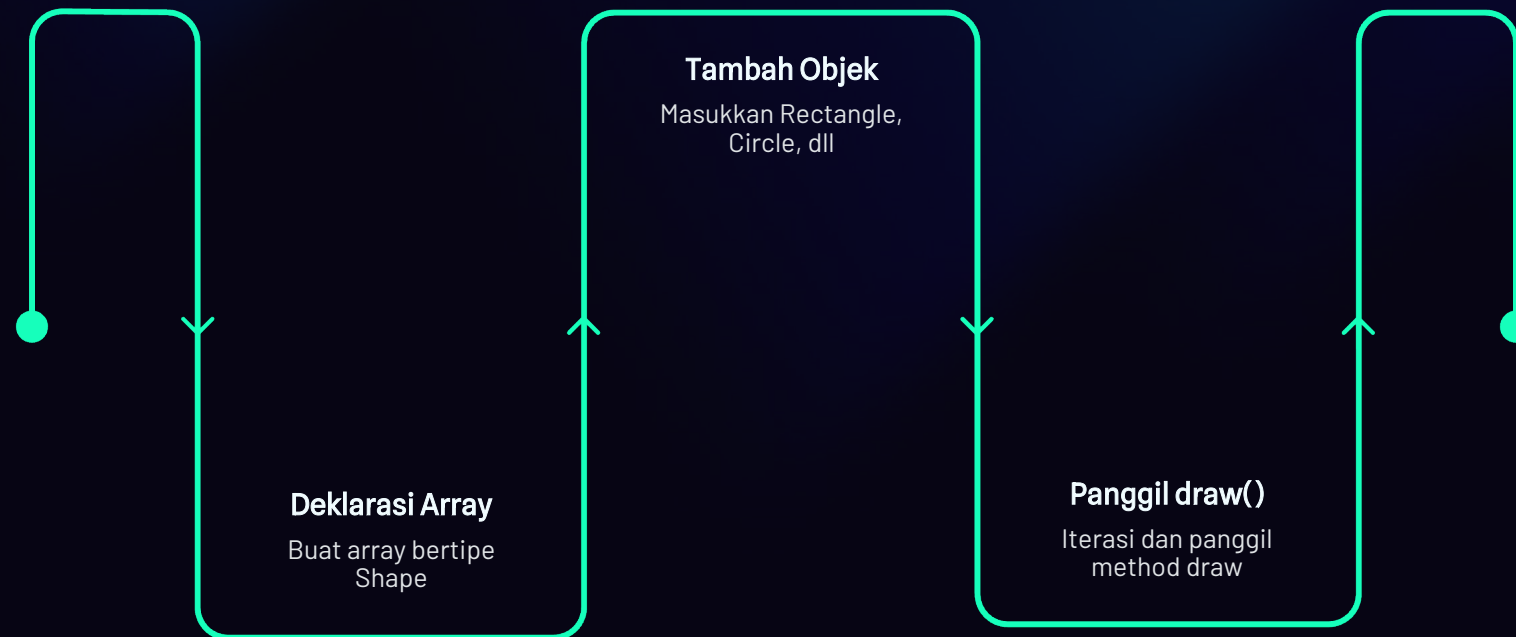
Polimorfisme sangat berguna dalam konteks grafis dan GUI. Berbagai bentuk dapat digambar menggunakan antarmuka yang seragam, memungkinkan sistem grafis yang fleksibel dan mudah diperluas.

Konsep Inti

Kelas **Shape** atau antarmuka **Drawable** mendefinisikan metode **draw()**. Setiap bentuk geometri – lingkaran, persegi, segitiga – mengimplementasikan **draw()** dengan logika gambar masing-masing.

Penerapan dalam Sistem Grafis

- Array `Shape[]` menyimpan semua bentuk pada kanvas
- Loop tunggal memanggil `draw()` untuk setiap elemen
- Java secara otomatis memanggil implementasi yang tepat
- Bentuk baru dapat ditambahkan tanpa mengubah kode pemrosesan



Terima Kasih

Semoga materi **Polymorphism and Interfaces** ini memberikan pemahaman mendalam tentang salah satu konsep terpenting dalam Pemrograman Berorientasi Objek.

Polimorfisme

Satu antarmuka, banyak implementasi. Kunci fleksibilitas dalam OOP.

Abstract Classes

Kerangka dasar hierarki yang tidak dapat diinstansiasi langsung.

Interface Java

Kontrak fleksibel yang terus berkembang sejak Java SE 8 dan 9.

Sasa Ani Arnomo, S.Kom., M.S.I.

Pemrograman Berorientasi Objek — Java
Universitas Putera Batam

